

## 职位链接

---

<https://cmbnt.cmbchina.com/pages/socialRecruit/detail.html?jobId=25AAA6FB967041778FEA695BCC7755DB&tType=1&isTop=0>

## 职位职责

---

- 1) 参与产品的需求分析，负责测试计划以及方案的制定、案例设计、测试执行、缺陷跟踪处理以及测试报告的编写工作，定期汇报项目进展情况；
- 2) 根据项目测试需要，使用自动化测试框架、平台及工具，开发自动化测试用例，涉及单元测试自动化、接口测试自动化及界面测试自动化；
- 3) 参与所负责产品的文档编写、自动化框架以及知识库建设等工作。

## 职位要求

---

- 1) 本科及以上学历，计算机相关专业；
- 2) 精通测试理论及测试用例设计、白盒黑盒测试技能；
- 3) 3年以上开发或测试工作经验，具备一定自动化测试经验，掌握常见自动化测试工具和自动化测试框架；
- 4) 熟练使用Java、Python中至少一种语言，熟悉MySQL、Oracle、SQL Server、DB2中至少一种关系型数据库；
- 5) 责任心强，有良好的合作精神，有较强的学习能力和沟通协调能力；
- 6) 具备以下一种或多种条件者优先考虑：
  - (1) 有手机自动化测试工具开发维护经验；
  - (2) 有成功的大规模自动化测试实施经验；
  - (3) 熟悉金融或者企业财务相关的业务，有参与金融系统测试经验者优先。

## 常见知识

---

微处理器有三个重要的性能指标，它们是：**时钟频率（主频）、字长和内存**

经济规律是经济现象和经济过程中**内在的、本质的、必然的**联系

项目总投资是指**固定资产投资与流动资金投资**之和

根据固定利率和浮动利率的资产和负债之间的不同组合，分为**正缺口**，即浮动利率资产占总比例 > 浮动利率负债占总比例；**负缺口**，即浮动利率资产占总比例 < 浮动利率负债占总比例；**零缺口**，即浮动利率资产占总比例 = 浮动利率负债占总比例。

GDP（国内生产总值）根据支出法分为**消费、投资、政府购买和净出口**等四大部分。

## 重排列表

<https://leetcode.cn/problems/reorder-list/description/>

```
class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        """
        Do not return anything, modify head in-place instead.
        """
        tmp = head
        mylist = list()
        while tmp != None:
            mylist.append(tmp)
            tmp = tmp.next

        j = len(mylist) - 1
        i = 0
        while i < j:
            mylist[i].next = mylist[j]
            i = i + 1
            if i == j:
                break

            mylist[j].next = mylist[i]
            j = j - 1

        mylist[i].next = None
```

## 问答题预测

### 1, 自我介绍 + 有没有了解银行科技的业务背景?

在面试准备过程中，我也梳理了目前银行科技的主要业务板块，我认为主要可以分为以下3个核心领域：1，核心银行业务系统：包括存款、贷款、支付结算等基础模块。对数据的强一致性和事务完整性要求较高。2，渠道整合与开放平台：随着移动互联网的发展，手机银行APP、微信银行等电子渠道成为主流。同时，银行通过API接口向第三方开放服务，这也增加了接口测试和安全测试的复杂性。3，风险管理与合规：包括反洗钱、欺诈检测、信用风险评估等。这类系统通常涉及复杂的规则引擎和大数据分析。最后，我的理解是，相比互联网业务追求的高并发和快速迭代，银行业务更看重稳定性、安全性和数据准确性。因此，在测试过程中，除了常规的功能验证，要特别关注资金安全、账务核对以及合规性检查。

### 2, 你了解测试流程吗？单测/集成测试/回归测试分别做什么？

在标准的软件开发周期中，测试流程通常贯穿始终。从需求分析阶段的需求评审，到开发阶段的单元测试、集成测试，再到系统构建后的系统测试、验收测试，最后是上线前的回归测试。

单元测试 (Unit Testing) 这是最底层的测试，通常由开发人员编写，也可以由测试人员辅助。它的目的是验证最小的可测试单元（如一个函数、一个类）是否能按预期工作。

集成测试 (Integration Testing) 当各个单元开发完成后，需要将它们组装在一起进行测试。集成测试关注的是模块间的接口交互和数据传递是否正确。

回归测试 (Regression Testing) 每当系统发生变更（如新增需求、修复Bug）时，我们需要重新执行已有的测试用例，以确保原有功能没有被破坏。

### 3，有成功的大规模自动化测试实施经验？

---

在九天数据处理引擎项目中主导**大规模自动化落地实施**，采用 Python+Pytest 搭建全链路接口自动化体系，累计落地 200 + 自动化用例；对接 Jenkins 接入 CI 流水线，代码提交触发自动回归，核心接口自动化覆盖率 90%，有效替代大量重复性手工回归，规避了参数联动失效、作业资源残留、非法参数导致任务阻塞等历史缺陷重复引入，完成引擎规模化自动化落地建设。

### 4，自动化测试用过哪些框架？

---

我项目中主要使用 Pytest 作为自动化测试框架。Pytest 轻量化且扩展性强，我主要利用它的 **Fixture 前置后置机制**统一管理测试环境、初始化测试数据，大幅精简重复代码。同时依托 Pytest 的**参数化、异常捕获、批量执行**特性，覆盖了引擎参数边界、多参数交互、异常入参、作业生命周期等复杂场景。最终基于这套能力完成了大规模自动化回归落地，有效保障九天数据处理引擎迭代质量

```

import pytest

# Fixture 前置后置机制
@pytest.fixture
def db_connection():
    print("\n连接数据库")
    conn = "database_connection"
    yield conn          # ← 这里交给测试用例使用
    print("\n关闭数据库")

def test_insert_data(db_connection):
    print(f"使用连接: {db_connection}")
    assert db_connection == "database_connection"

# 参数化测试
@pytest.mark.parametrize("username, password, expected", [
    ("user1", "pwd1", "ok"),
    ("user1", "wrong", "fail"),
    ("", "", "fail"),
])
def test_login(username, password, expected):
    result = login(username, password)
    assert result == expected

# 异常捕获
def divide(a, b):
    if b == 0:
        raise ValueError("除数不能为 0")
    return a / b

def test_divide_zero():
    with pytest.raises(ValueError):
        divide(10, 0)

```

Fixture比传统的 setup/teardown 更灵活，支持作用域控制（function / module / session），还可以通过 yield 实现自动清理。

## 5, Git常用命令？团队协作怎么做版本管理？

### Git 常用命令

1. git clone：拉取远程仓库代码
2. git pull：拉取最新代码，同步远程更新

3. `git add`: 添加文件到暂存区
4. `git commit -m"注释"`: 提交代码到本地仓库
5. `git push`: 推送到远程仓库
6. `git branch`: 查看 / 创建 / 删除分支

```
git branch          # 查看当前分支
git branch dev      # 创建 dev 分支
git checkout dev    # 切换到 dev
# 开发、提交代码
git checkout main   # 切回 main
git merge dev       # 合并 dev
git branch -d dev    # 删除 dev
```

7. `git checkout`: 切换分支
8. `git merge`: 合并分支代码
9. `git status`: 查看代码状态
10. `git log`: 查看提交记录

## 版本管理

团队采用分支规范化管理，master 保持稳定、develop 用于集成、feature 开发新功能，开发完成后通过代码评审合并，定期 `git pull` 拉取最新代码避免冲突，保证版本干净、可追溯、稳定上线。

## 6，索引类型 + SQL 优化

索引方面，我主要使用主键索引、唯一索引、普通索引和联合索引，其中联合索引要遵循最左前缀原则。

SQL 优化方面，我会优先保证 WHERE、ORDER BY、JOIN 字段有合适索引，避免对索引字段做函数运算、使用负向查询和前模糊匹配，同时通过 EXPLAIN 分析执行计划，减少扫描行数和临时表使用。

## 7，反问环节

想了解一下目前项目的迭代节奏大概是怎样的？日常测试是以版本集中迭代测试为主，还是比较偏向常态化分批交付、持续集成的模式？

咱们目前自动化测试的覆盖重点是接口自动化、UI 自动化还是单元测试？目前核心业务链路自动化覆盖率大概在什么水平？

## 数据库刷题：

```
select tweet_id
from Tweets
where char_length(content) > 15;
```

```
select a.id
from Weather a inner join Weather b
on datediff(a.recordDate,b.recordDate) = 1
where a.Temperature > b.Temperature;
```

```
select a.machine_id,round(avg(b.timestamp-a.timestamp),3) as processing_time
from Activity a inner join Activity b
on a.machine_id = b.machine_id and a.process_id = b.process_id and a.activity_type =
"start" and b.activity_type = "end"
group by a.machine_id;
```

```
select a.student_id,a.student_name,b.subject_name, count(e.subject_name) as
attended_exams from
Students a cross join Subjects b
left join Examinations e
on a.student_id = e.student_id and b.subject_name = e.subject_name
group by a.student_id,a.student_name,b.subject_name
order by student_id,subject_name;
```

```
select a.user_id, round(IFNULL(AVG(b.action='confirmed'),0),2) as confirmation_rate
from Signups a left join Confirmations b
on a.user_id = b.user_id
group by a.user_id;
```

```
select *
from cinema
where description <> "boring" and id % 2 = 1
order by rating desc;
```

```

select a.product_id, ifnull(round(sum(a.price * b.units)/sum(b.units),2),0) as
average_price
from Prices a left join UnitsSold b
on a.product_id = b.product_id and b.purchase_date between a.start_date and a.end_date
group by a.product_id;

```

```

select contest_id, round(count(user_id) * 100 /(select count(*) from Users),2)
percentage from
Register group by contest_id
order by percentage desc,contest_id;

```

```

select DATE_FORMAT(trans_date, '%Y-%m') month,
country,
count(*) trans_count,
sum(state="approved") approved_count,
sum(amount)trans_total_amount,
sum(if(state="approved", amount,0)) approved_total_amount
from Transactions
group by month,country;

```

```

select round(count(b.event_date2)/count(distinct a.player_id),2) fraction
from Activity a left join
(select player_id,min(event_date) as event_date2 from Activity group by player_id) b
on a.player_id = b.player_id and datediff(a.event_date,b.event_date2)=1;

```

```

select player_id,DATE_ADD(MIN(event_date), INTERVAL 1 DAY)

```

```

select customer_id
from Customer
group by customer_id
having count(distinct product_key)=(select count(*) from Product);

```

```

select employee_id, department_id
from Employee
where primary_flag = 'Y'
or employee_id in (
    select employee_id
    from Employee
    group by employee_id
    having count(*) = 1
);

```

```

select employee_id,
if(count(department_id) = 1, department_id, max(if(primary_flag='Y', department_id,
null))) as department_id
from Employee
group by employee_id;

```

```

SELECT
    x,
    y,
    z,
    CASE
        WHEN x + y > z AND x + z > y AND y + z > x THEN 'Yes'
        ELSE 'No'
    END AS 'triangle'
FROM triangle;

```

```

select user_id, concat(upper(substring(name,1,1)),lower(substring(name,2))) name
from Users
order by user_id;

```

```

select patient_id,patient_name,conditions
from Patients
where conditions like "DIAB1%" or conditions like "% DIAB1%";select *

```